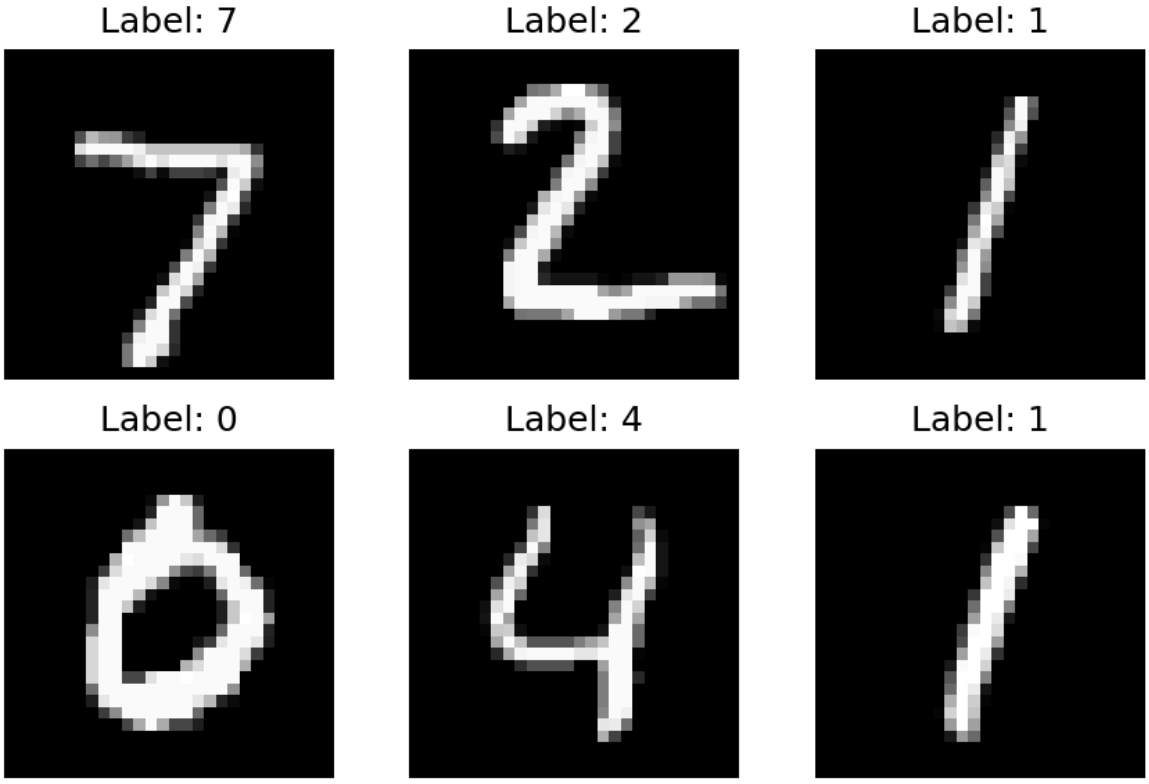
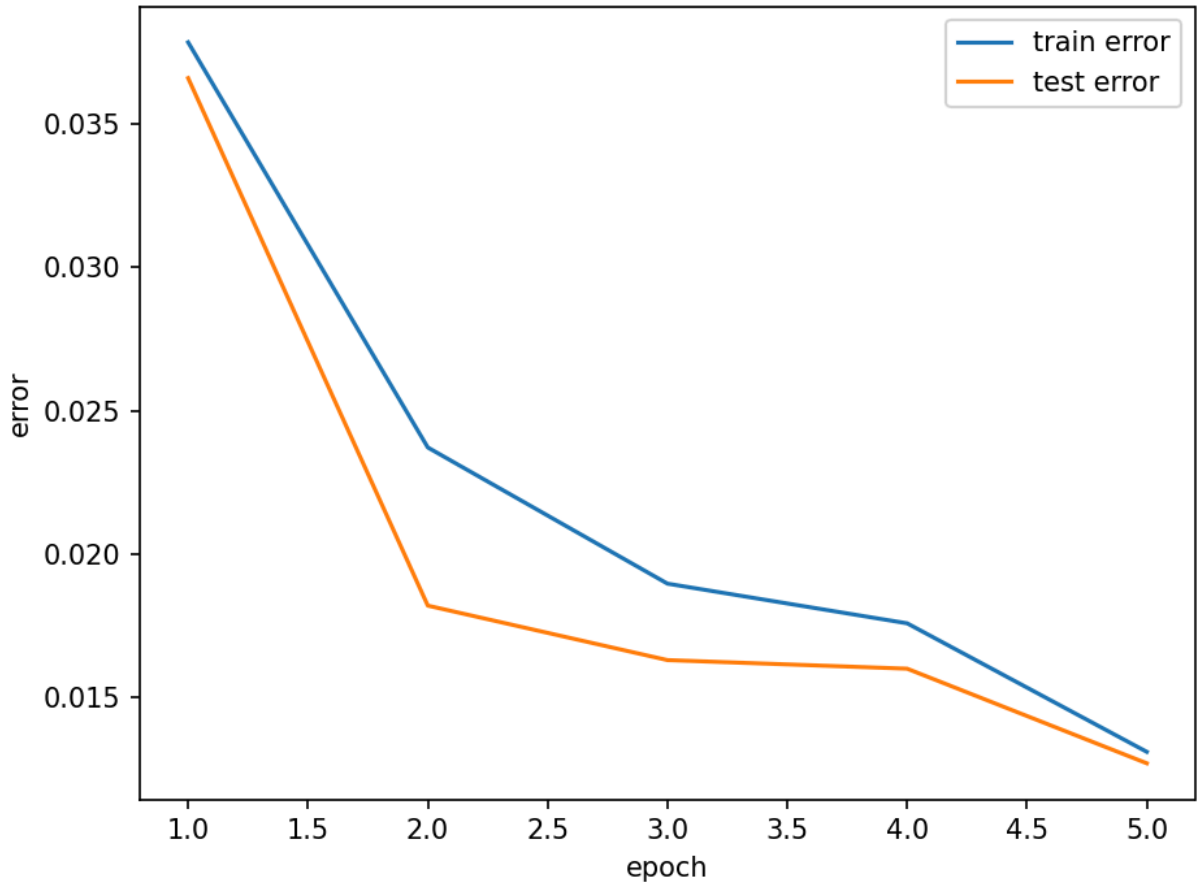
Dana Tran
CS 5330 Project 5
04/07/26

Description
This project was about building, training, and then opening up a deep network to see what it learned. I trained a small convolutional network in PyTorch on MNIST digits, saved the trained weight and then reused the saved model for the follow up tasks. After training, the network runs on fixed test examples and printed the full output vector for each sample so I could see both the prediction and the model's confidence. Lastly, I used transfer learning by freezing the MNIST feature extractor, replacing the final classification layer, and retraining only that last layer to classify Greek letters. All in all, the assignment connected the training of a model with the understanding of the model.

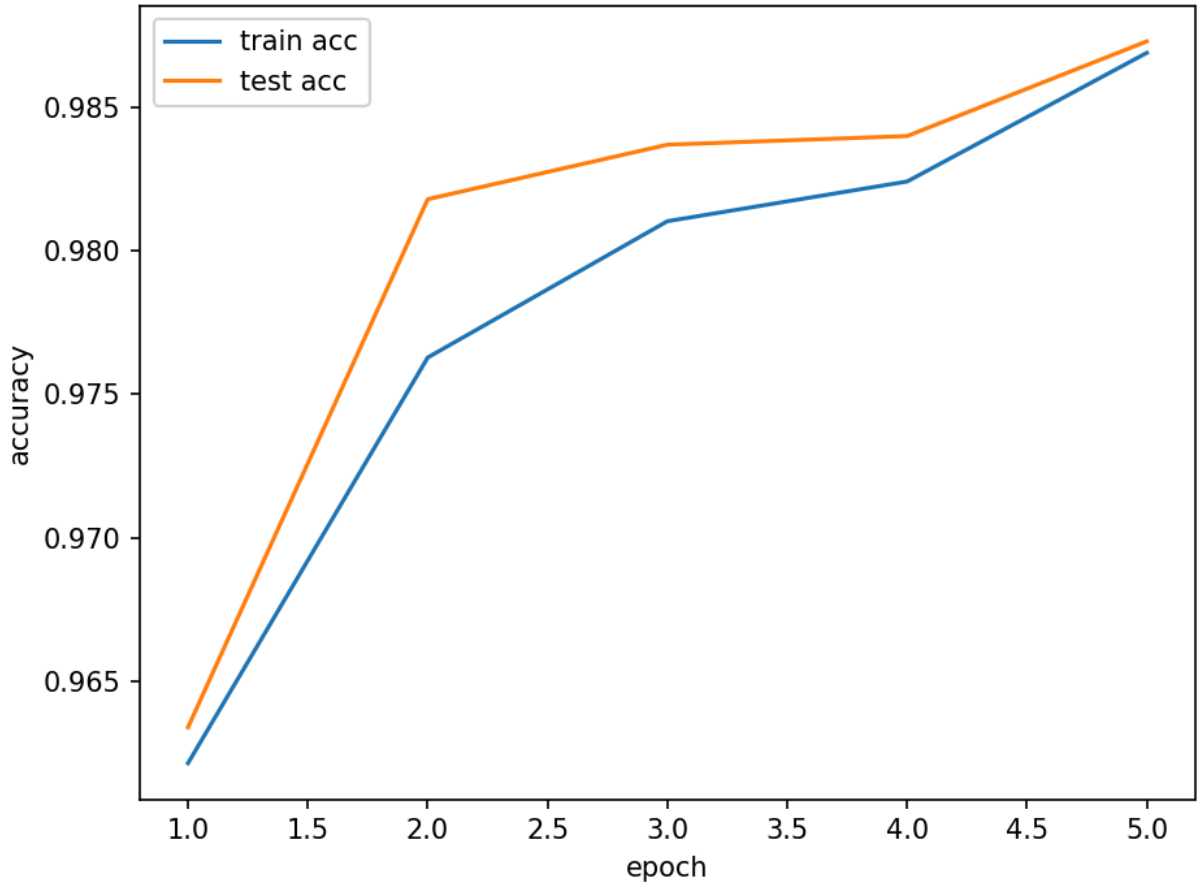
Images
First 6 MNIST test digits 2x3 grid - Shows the first six images from the MNIST test set in a fixed order with their labels.



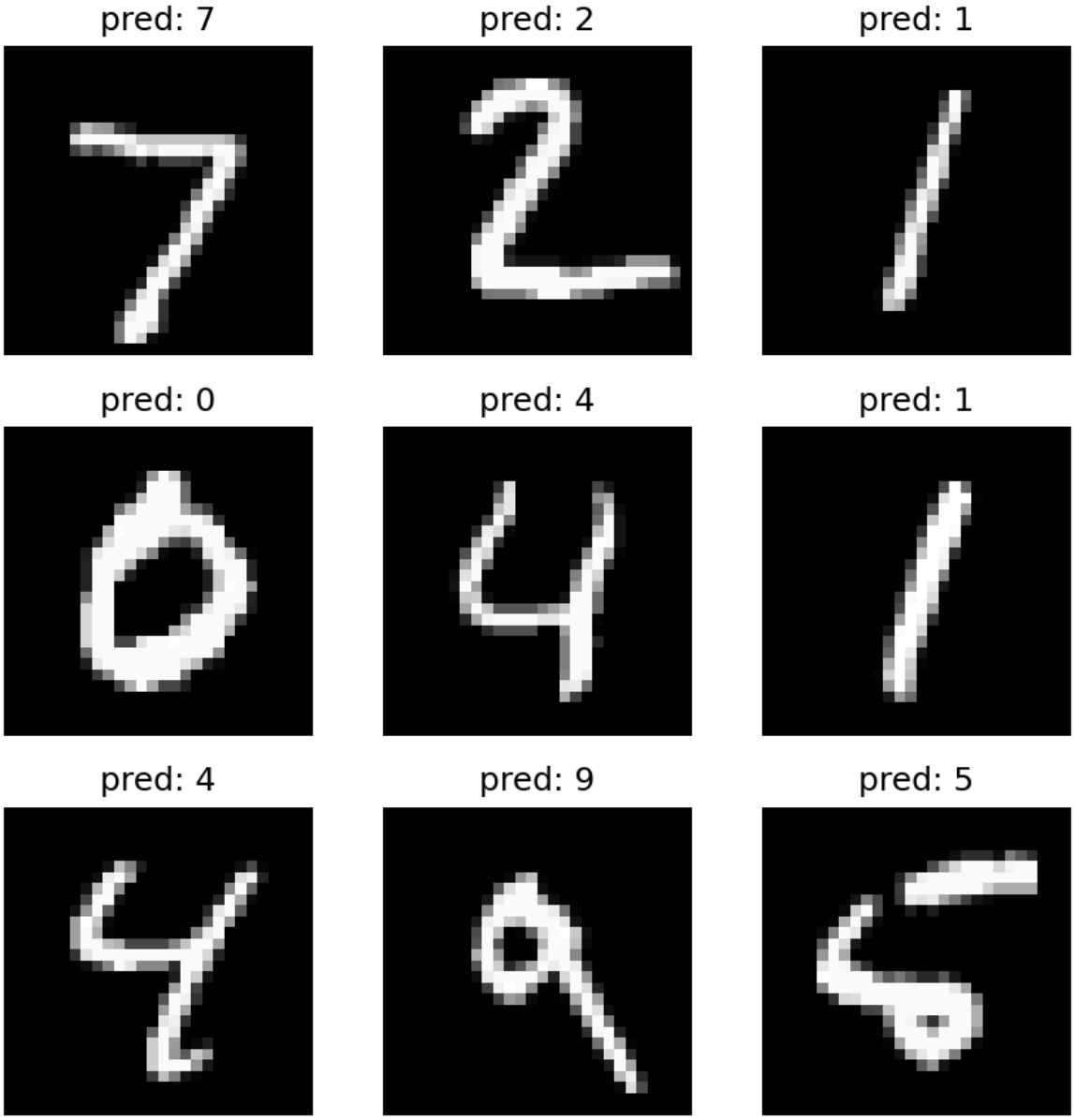
Training/testing error curve CNN - Error per epoch for both the training and test sets. This shows CNN improving over epochs.


train error
test error
0.035
0.030
0.025
0.020
0.015
error
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0
epoch

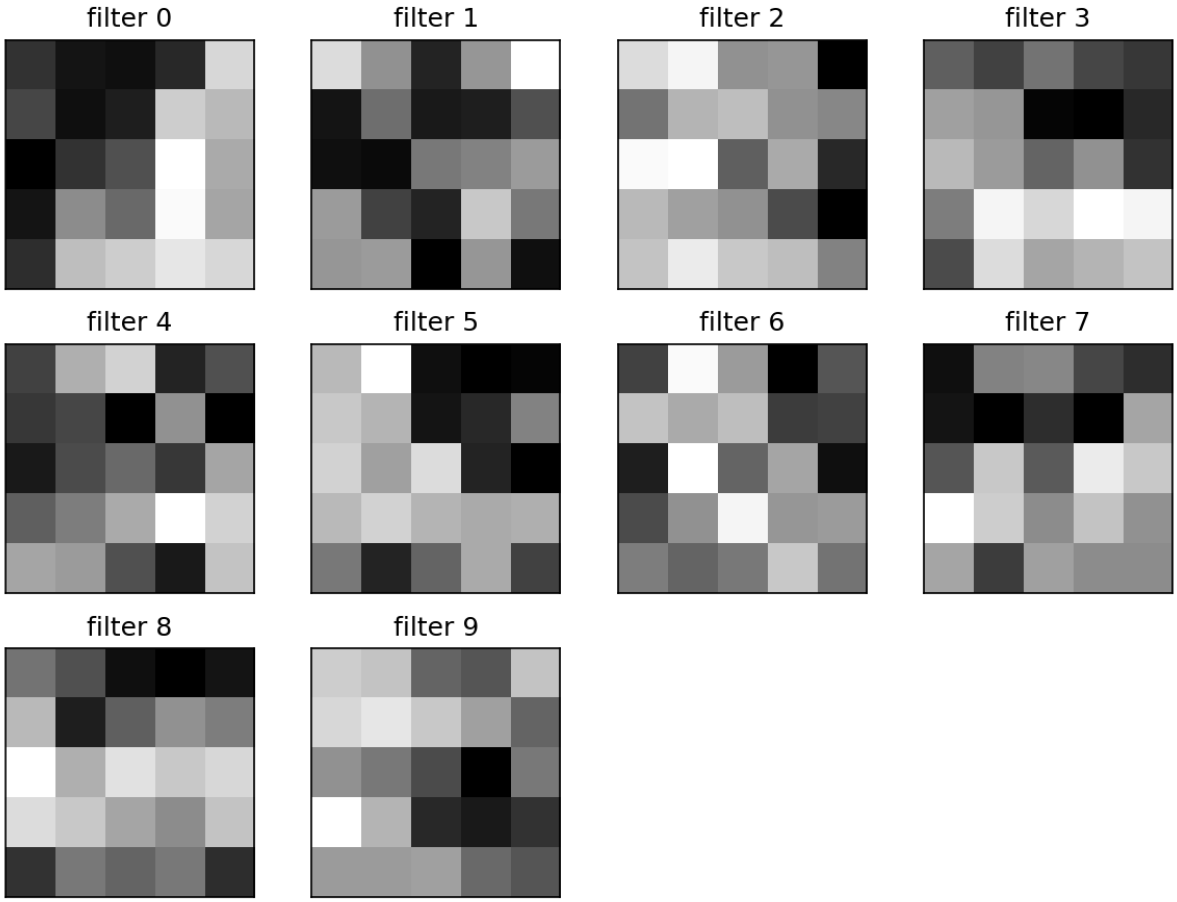
Training/testing accuracy curve CNN - Accuracy per epoch for both training and test sets. This makes it easy to compare generalization.



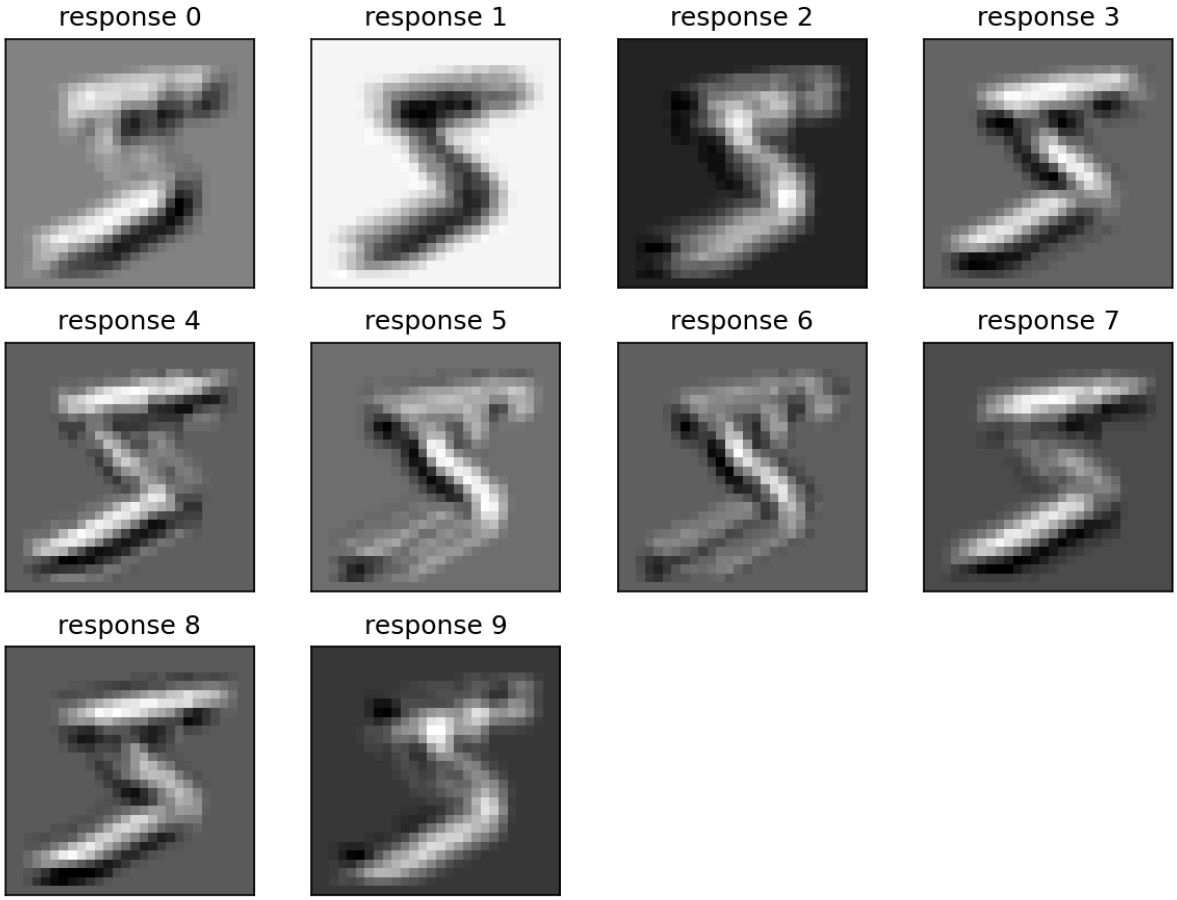
First 9 MNIST test predictions 3x3 grid - Shows the first nine test images with the predicted label above each digit



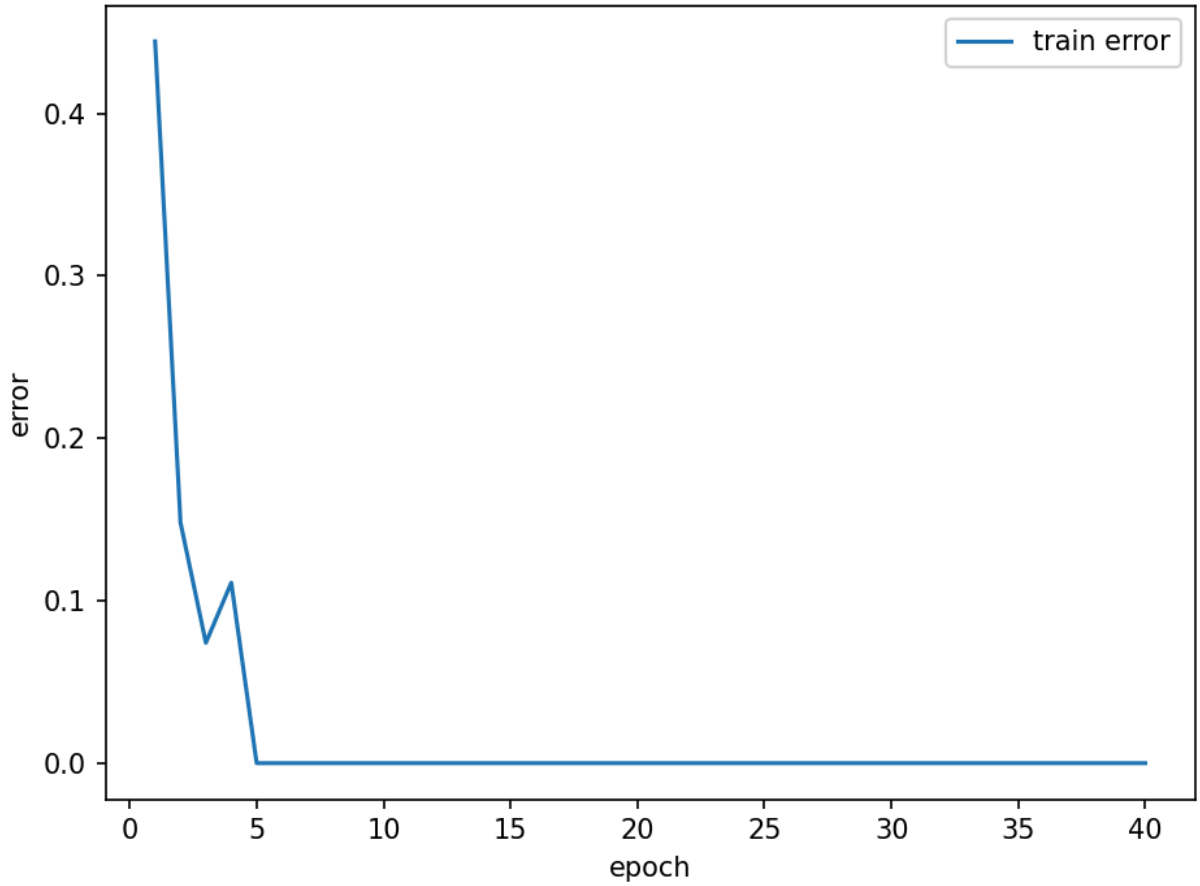
conv1 filters visualization 10 filters - The 10 learned 5x5 filters in the first convolution layer that are the actual weights the network learned.



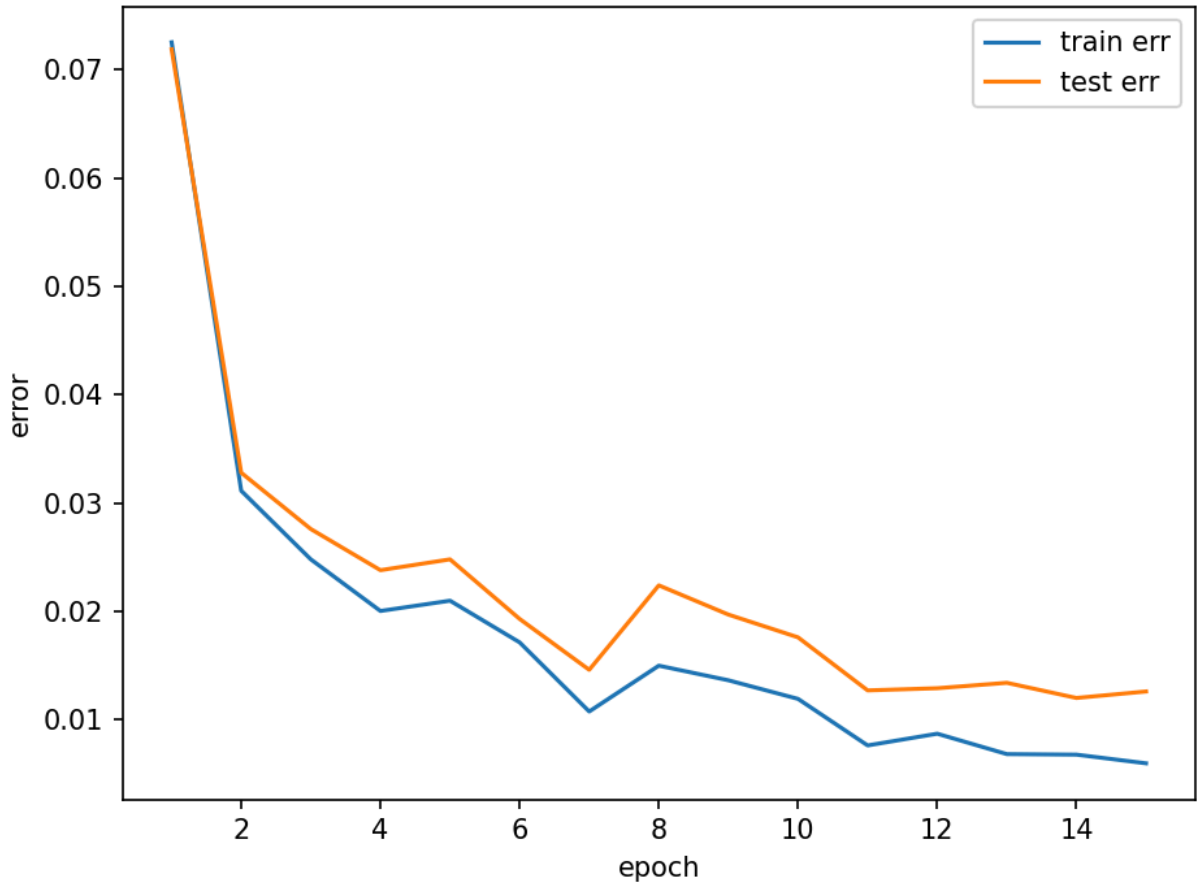
conv1 filter results on a sample image - The result of applying each conv1 filter to the first MNIST training image using filter2D. This shows which structures each filter responds to.

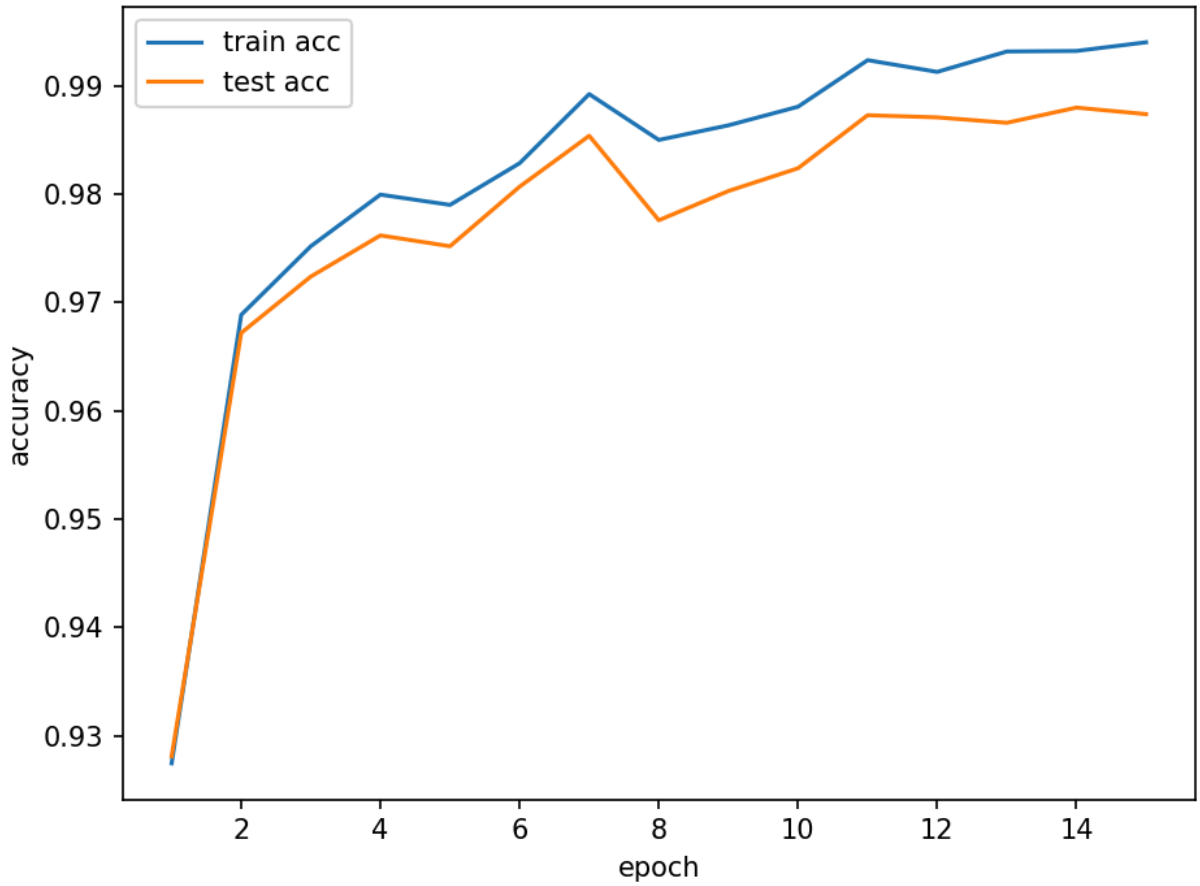

response 0
response 1
response 2
response 3
response 4
response 5
response 6
response 7
response 8
response 9

Greek transfer learning training error - Training error vs epoch on the 27 Greek letter images (alpha/beta/gamma). Look how quick the modified network fits the new classes.



Transformer curves - Error/accuracy curves for the transformer-based model using the template's default settings.

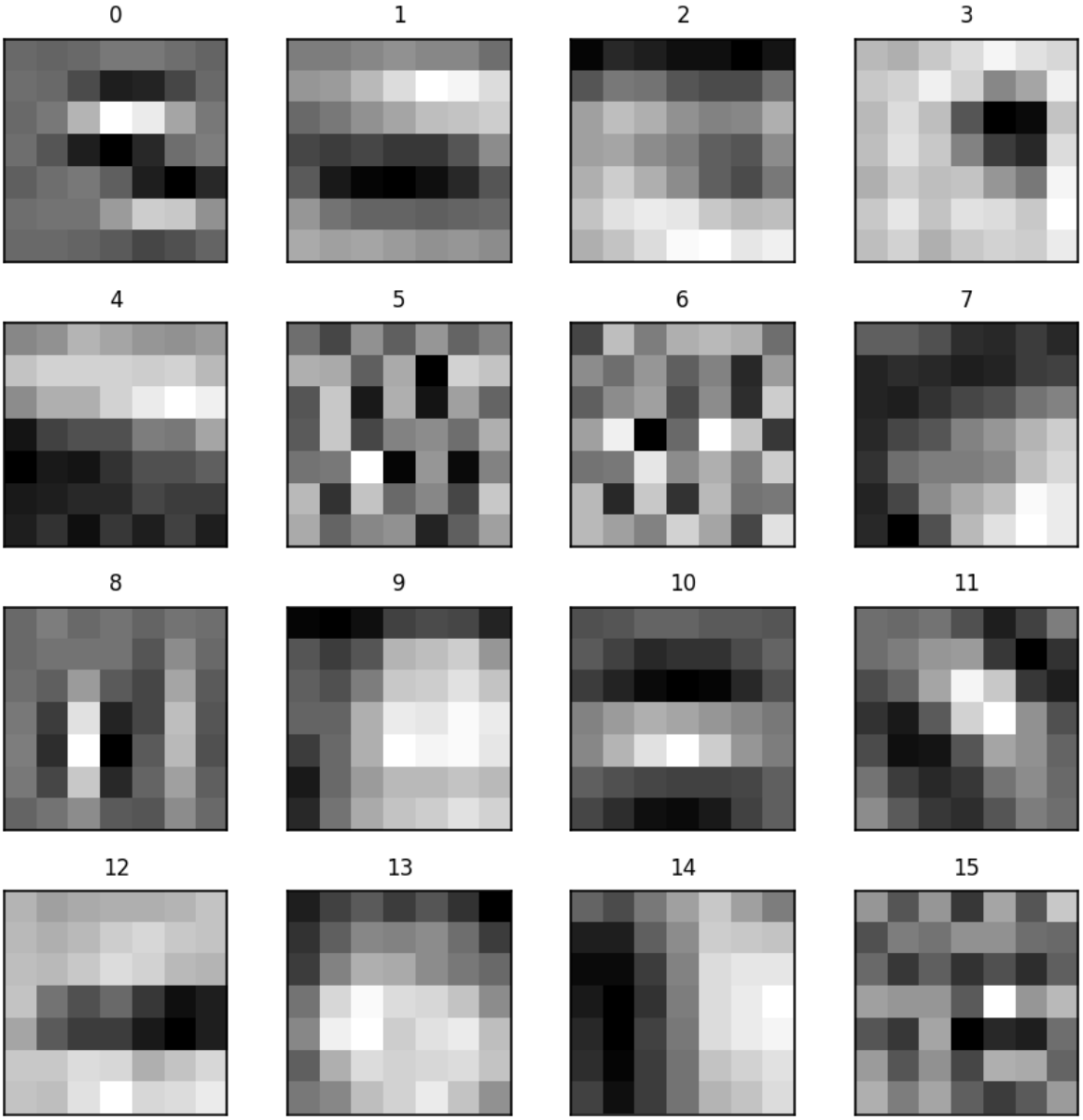

train err
test err
0.07
0.06
0.05
0.04
0.03
0.02
0.01
error
2
4
6
8
10
12
14
epoch



Extensions

1. load a pretrained network and inspect early conv layers - I loaded pretrained ResNet18 weights from torchvision and inspected its early convolution layers like what we did for Task 2. I printed the model structure, printed the shape of the first conv weights, and

saved a grid visualization of conv1 filters.



2. Live video digit recognition - Built a small OpenCV webcam demo that takes a live frame, converts it to grayscale, centers, crops it, resizes it to 28x28, applies the MNIST normalization, runs the trained MNIST model, and overlays the predicted digit on the video.

Reflections

Training a model is really only the first step, making it usable and understandable matters just as much. Saving/loading models, switching between train() and eval() for dropout, and matching input preprocessing are all real important for getting consistent results. Visualizing conv1

weights and their responses made the network feel less like a black box: even simple 5x5 filters start to form useful edge detectors. Transfer learning was the biggest surprise with only 27 Greek letter images, freezing the network and retraining the last layer worked quickly. The handwritten digit test also showed that real world inputs can break a model unless you have more control over preprocessing and overall data style. This project also tested my patience a bit once I got to the next task because the first epochs took awhile to run.

Acknowledgements
PyTorch / torchvision documentation and tutorials
Transformer template and GreekTransform snippet
OpenCV documentation for filter2D and webcam capture
W3schools to refresh myself on python